



Whispy: Ein Python-Framework für Hörversuche

Aron Ventura, Maximilian Will, Tom Strobl | Python & Akustik SS26 | 13. Juli 2026

Aufgabenstellung

Entwicklung eines Python-basierten Hörversuchsprogramms

- Implementierung als zuverlässiges Versuchsprogramm welches mit vorgerenderten Dateien arbeitet und zwingend ASIO-Treiber unterstützt
- Python-Programm mit Frontend (GUI) zur Versuchsdurchführung und Backend zur Versuchssteuerung
- Unterstützung mehrerer Versuchsverfahren, erweiterbar um zukünftige Verfahren (z. B. Ratingskala, N-AFC Staircase, Paarvergleich)
- Plattformübergreifende Einsetzbarkeit unter Windows und macOS sowie einfache, schnelle Inbetriebnahme

Anforderungen

Muss

- ✓ Intuitive, anwenderfreundliche GUI für Probanden
- ✓ ASIO-kompatible Wiedergabe von Audiosignalen + Auswahl und Verwaltung von Stimuli
- ✓ Verfahren: N-AFC Staircase-Verfahren / AB-X Test
- ✓ Erweiterbarkeit für zusätzliche Verfahren
- ✓ Erfassung aller Antworten und Metadaten / Speicherung im .csv Format

Anforderungen

Kann

- ✓ Open Source Software als Git-Repository
- ✓ Implementierung eines OSC-Handlers
- ✓ Implementierung von frei konfigurierbaren Fragebögen + nützlichen Beispielen
- ✓ Implementierung von verschiedenen Scale-Testing Verfahren
- ✓ Anonymisierte Datenspeicherung
- ✓ Plot-Modul, um Ergebnisse direkt nach dem Experiment zu visualisieren

Umsetzung – Ansatz & Technologien

Modularer Python-Stack, orchestriert über Jupyter-Notebooks

- Sprache & Umgebung: Python, Jupyter Notebooks als Experiment-Driver
- Konfiguration: zentrale YAML-Dateien für Design-, Frage- und Testparameter
- GUI: PyQt6 für alle Interaktionsfenster (Drag&Drop, N-AFC, ABX, Scale, Fragebogen, Info)
- Stimulus: SoundDevice-Audio-Wiedergabe oder OSC-Messages (Python-osc)
- Daten: Ergebnisse als Pandas DataFrame/CSV + Visualisierung via Plots (show and save)
- Versionierung: Git/GitHub mit automatisch gerendertem Mermaid-Architekturdiagramm
- Weitere Software: PyCharm, Claude Code, VS-Code

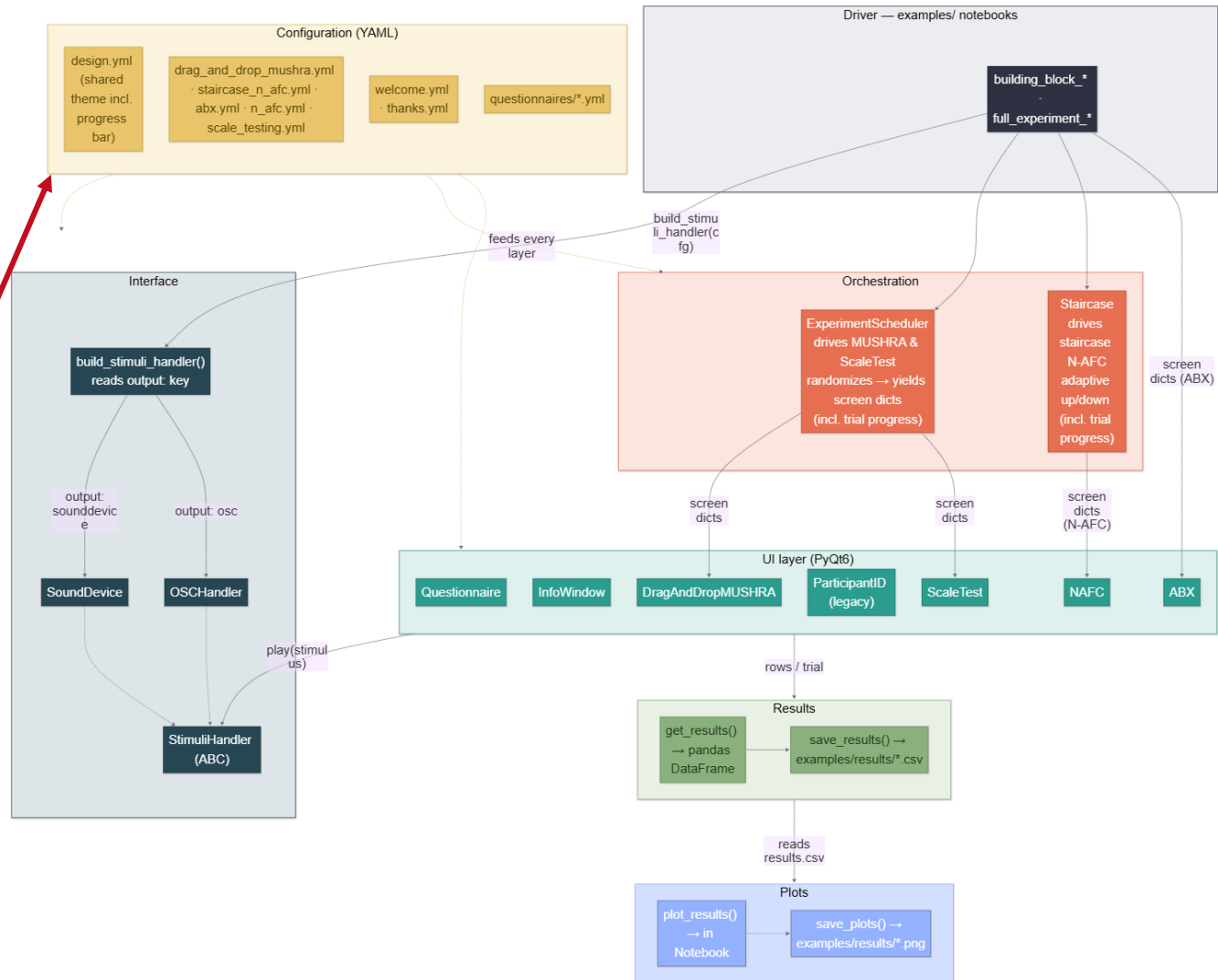
Umsetzung – Architektur

Configuration

Zentrale YAML-Dateien für:

- Design
- Output
- Fragebögen
- Testparameter

➡ was gezeigt wird



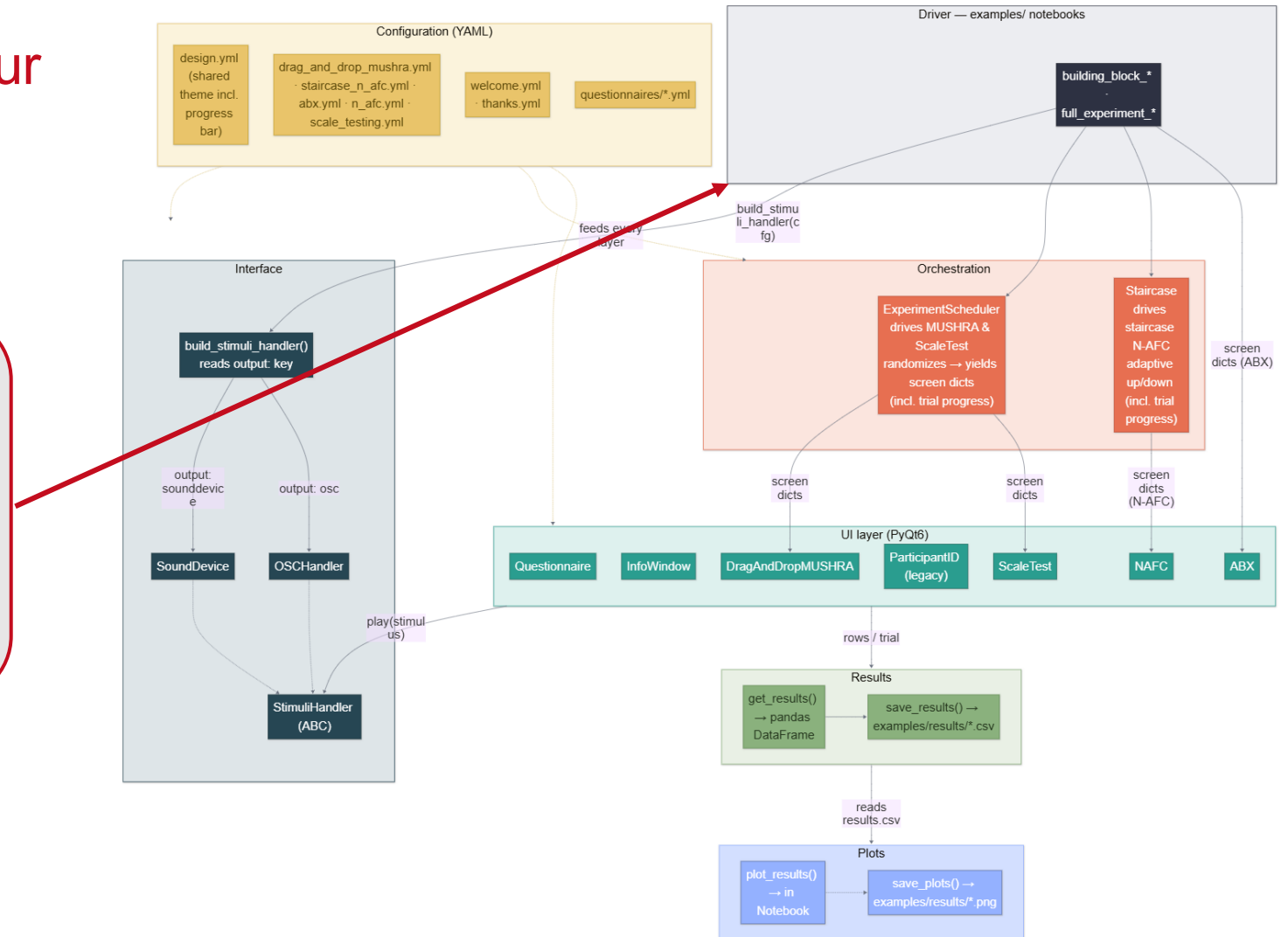
Umsetzung – Architektur

Driver

Jupyter-Notebooks orchestrieren entweder:

- Bausteine (Experimente selbst zusammenstellen), oder
- Komplette Experimente, fertig ausführbar

➔ was wird gemacht

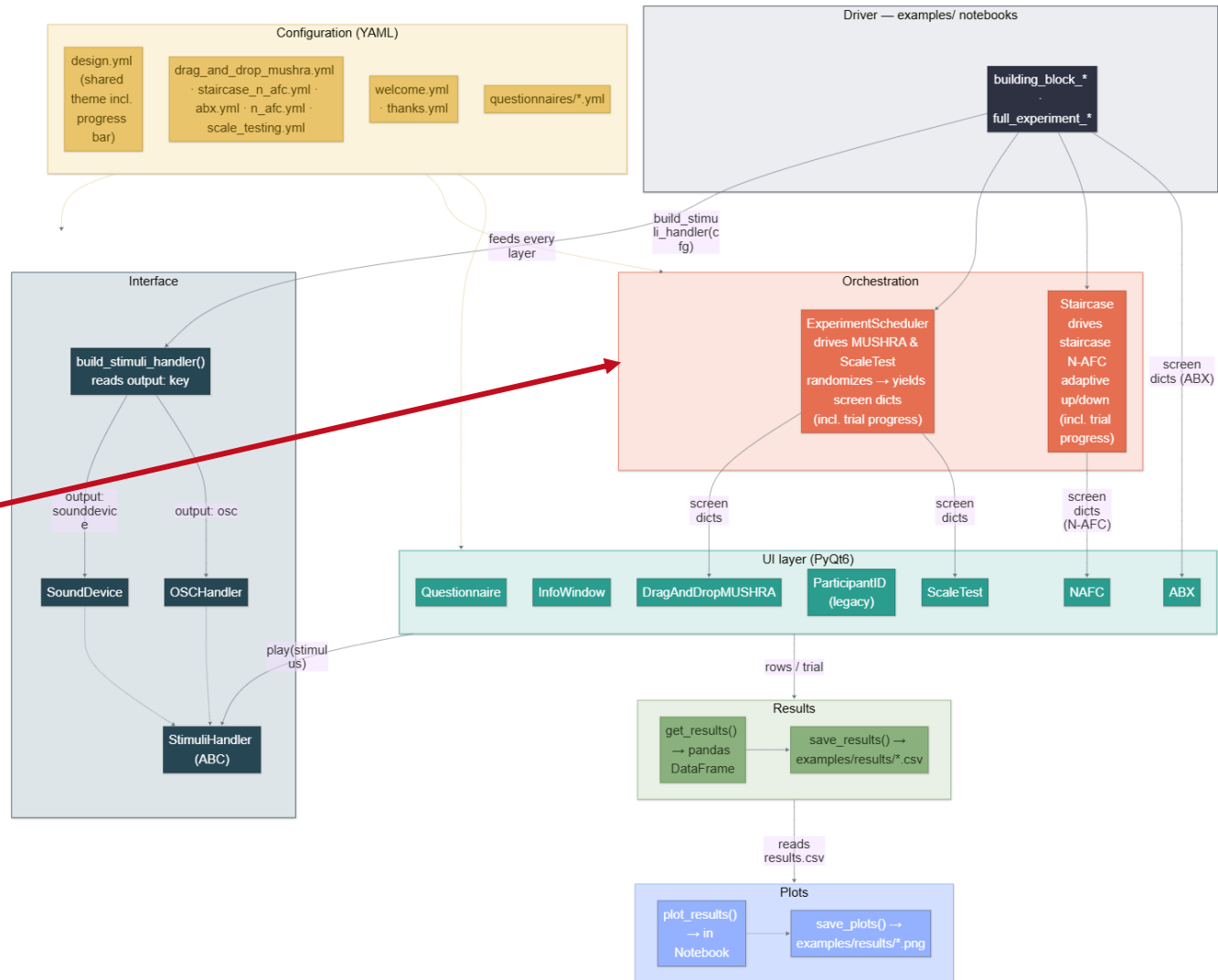


Umsetzung – Architektur

Orchestration

- Testablauf und Logik für
 - ExperimentScheduler (MUSHRA, Scale Testing)
 - Staircase (adaptives N-AFC)

➔ in welcher Reihenfolge



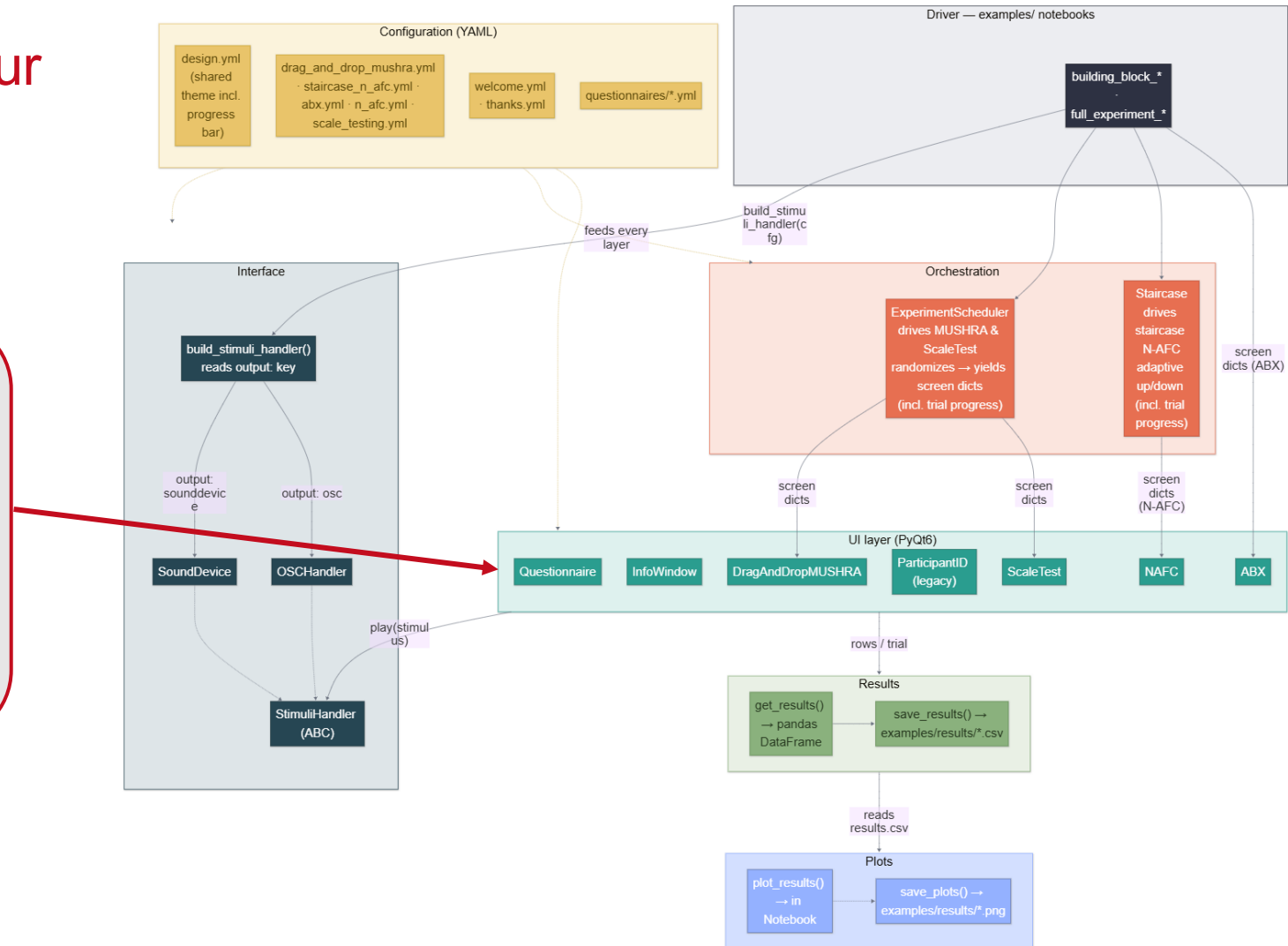
Umsetzung – Architektur

UI-Layer (PyQt6)

Fenster für

- Drag&Drop-MUSHRA
- N-AFC
- ABX
- Scale Testing
- Fragebogen
- Info

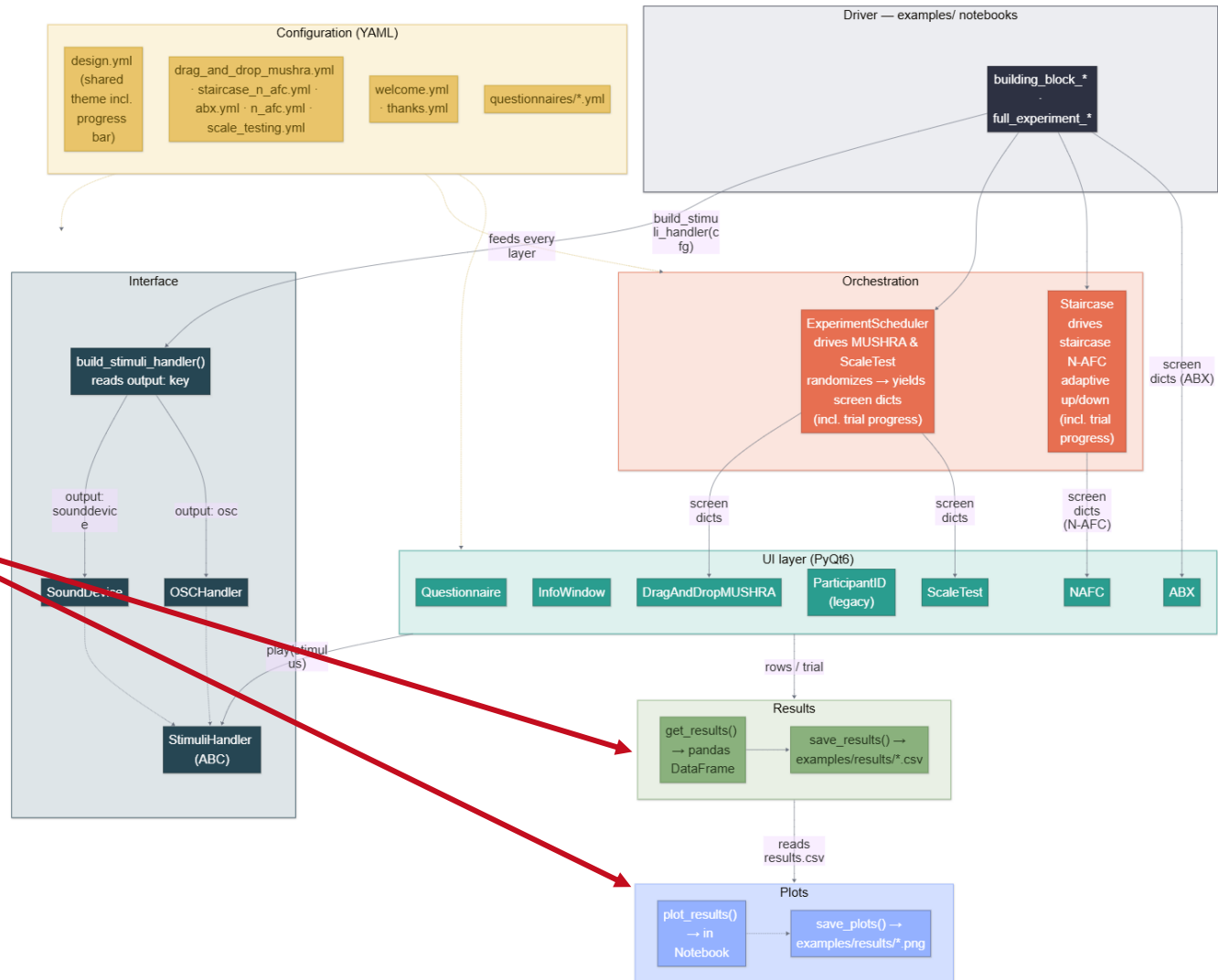
➔ wie es aussieht



Umsetzung – Architektur

Results

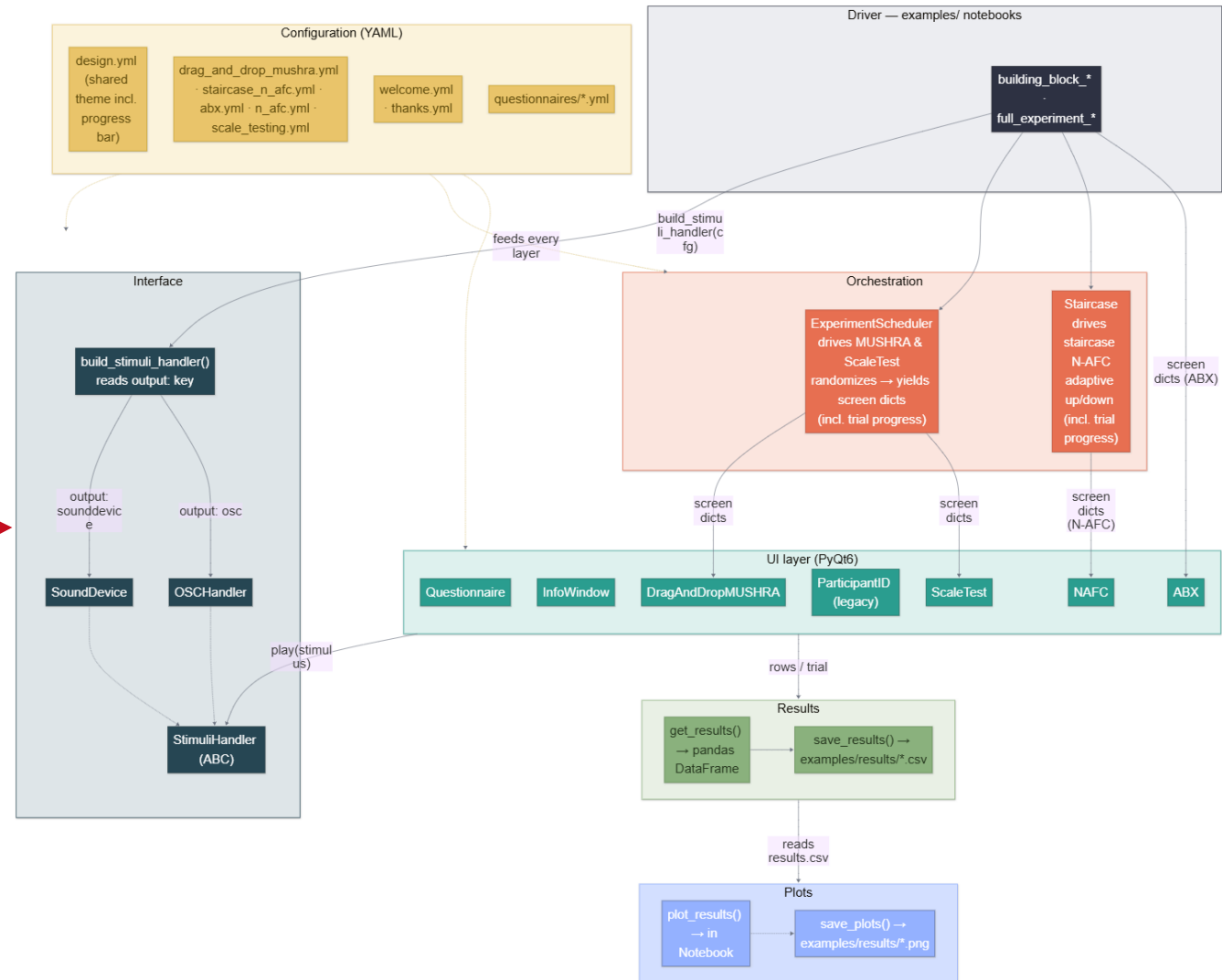
- **get_results()** liefert DataFrame
- **save_results()** speichert als CSV
- **plot_results()** zeigt Visualisierung der Ergebnisse (Reaktionszeit, Antwortgenauigkeit, ...)
- **save_plots()** speichert die Plots für einfache Verwendung
➔ Ergebnisverwaltung



Umsetzung – Architektur

Interface

- Factory.py
 - Mit build_stimuli_handler() -> entscheidet welcher Stimuli Handler gebaut wird
- StimuliHandler-Basisklasse
 - SoundDevice (Audio Ausgabe)
 - OSC-handling (Fernsteuerung)
 - ➔ Backendverwaltung

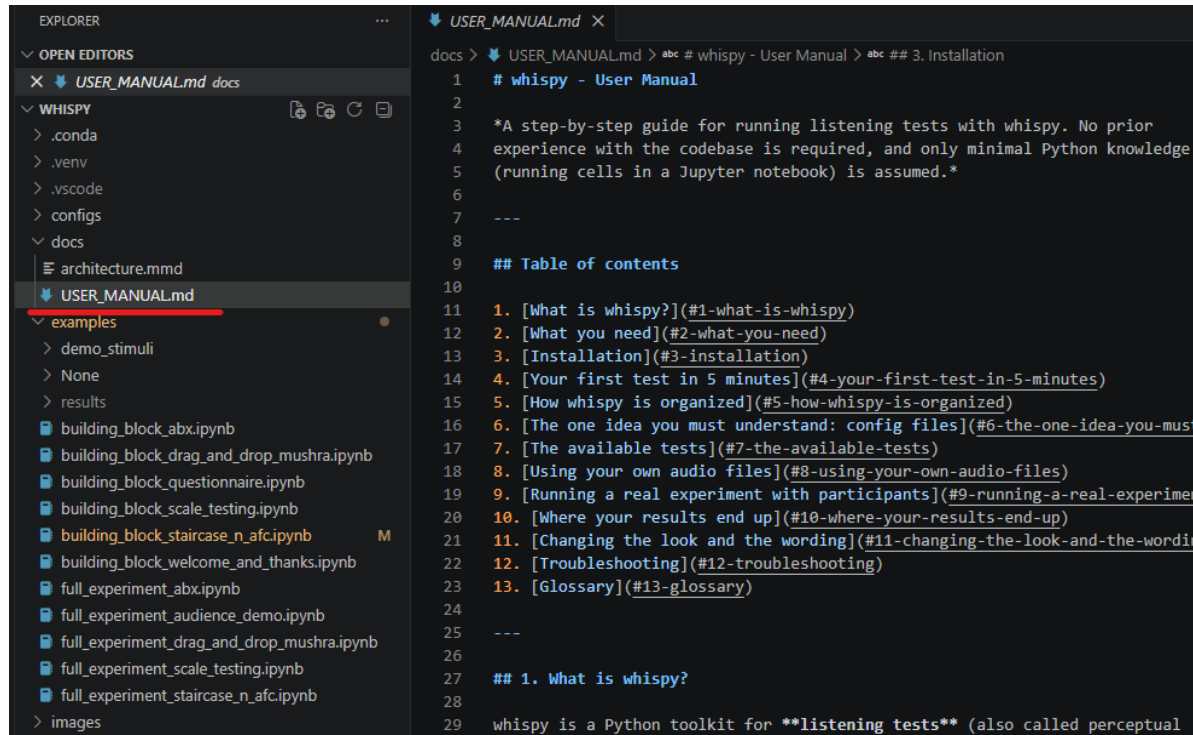


User Workflow

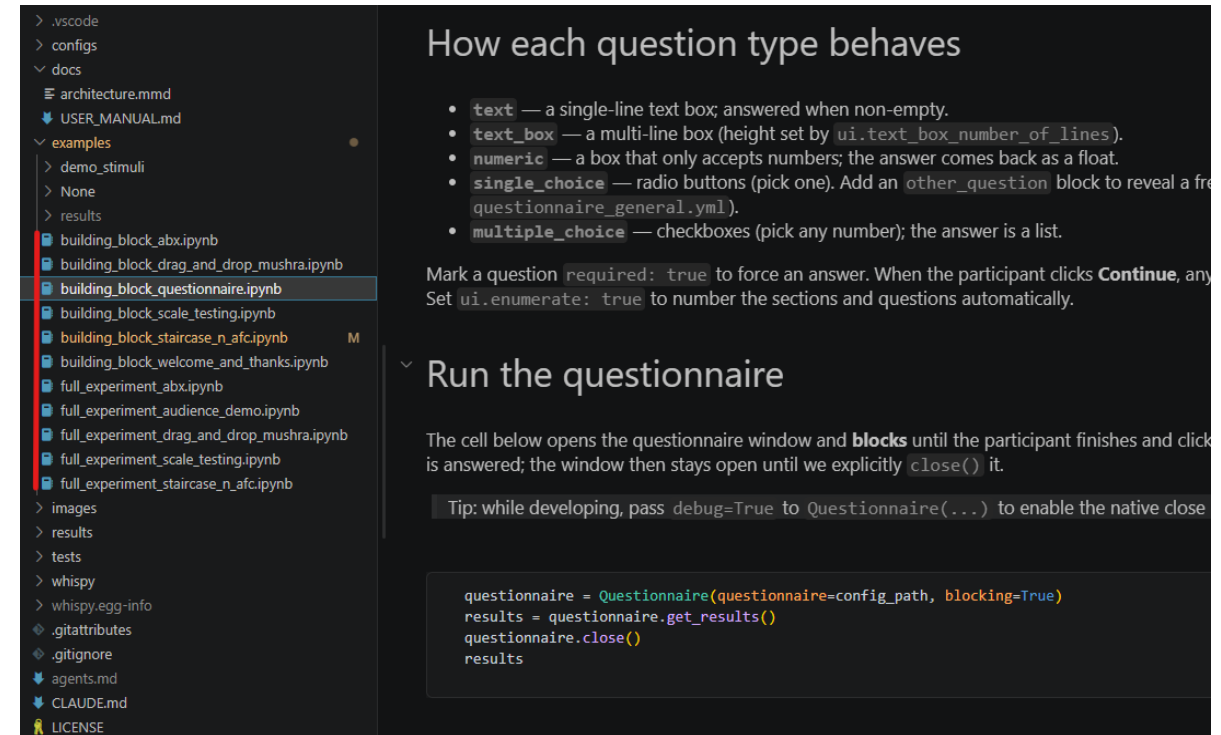


Dokumentation

Benutzeranleitung und Jupyter-Notebooks



The screenshot shows the VS Code Explorer on the left with the 'USER_MANUAL.md' file selected under the 'docs' folder. The main editor displays the content of 'USER_MANUAL.md', which is a user manual for 'whispy'. It includes a table of contents with 13 items, such as 'What is whispy?', 'What you need', 'Installation', and 'Your first test in 5 minutes'. The file is structured with markdown headers and a list of links to various Jupyter notebooks.



The screenshot shows the VS Code Explorer on the left with the 'building_block_questionnaire.ipynb' file selected under the 'examples' folder. The main editor displays a text document titled 'How each question type behaves'. It lists several question types and their behaviors:

- text** — a single-line text box; answered when non-empty.
- text_box** — a multi-line box (height set by `ui.text_box_number_of_lines`).
- numeric** — a box that only accepts numbers; the answer comes back as a float.
- single_choice** — radio buttons (pick one). Add an `other_question` block to reveal a free text box (set by `questionnaire_general.yml`).
- multiple_choice** — checkboxes (pick any number); the answer is a list.

Below the list, it explains how to mark a question as required and how to set `ui.enumerate` to number sections and questions automatically.

Run the questionnaire

The cell below opens the questionnaire window and **blocks** until the participant finishes and clicks 'Continue'; the window then stays open until we explicitly `close()` it.

Tip: while developing, pass `debug=True` to `Questionnaire(...)` to enable the native close button.

```
questionnaire = Questionnaire(questionnaire=config_path, blocking=True)
results = questionnaire.get_results()
questionnaire.close()
results
```

Code Demo

Three small, colored circles (red, yellow, green) are positioned above the text, mimicking the window control buttons of a standard operating system.

```
full_experiment_audience_demo.ipynb
```

Fragen & Diskussion

Vielen Dank für die Aufmerksamkeit, wir freuen uns auf die Diskussion

